Computing for Analytical Work

Session 2 — What is running?

Block 4 — Notebooks, scripts, state, and reproducibility

This block in one sentence

A notebook is an interface to a running Python process with hidden state. A script starts fresh each time.

Hidden state is why a notebook can look finished and be broken.

What we cover

- The page and the kernel: two things, one screen
- What the kernel remembers, with a prediction
- The three ways that memory lies: order, leftovers, saved outputs
- Restart Kernel and Run All — the execution test, and what it does not test
- Notebook vs script; minimal reproducibility
- Where the kernel comes from: the rule, again
- Git thread: commit messages, checkpoints you can return to, `.gitignore`
- AI thread, stretch, Lab 4, and Homework 2

What a notebook actually is

The page (.ipynb)

[1]

```
df = pd.read_csv("data/raw/sales.csv")
```

[2]

```
total = df["revenue"].sum()
28317.25
```

The kernel (a running process)

what it remembers right now

```
df = <DataFrame 35 rows>
```

```
total = 28317.25
```

Run cell 1, then cell 2: the kernel now holds df and total. The bracket is the order it ran in.

Delete a cell. The kernel does not notice.

The page (.ipynb)

[1]
```
df = pd.read_csv("data/raw/sales.csv")
```
deleted from the page

[2]
```
total = df["revenue"].sum()
28317.25
```

The kernel (a running process)

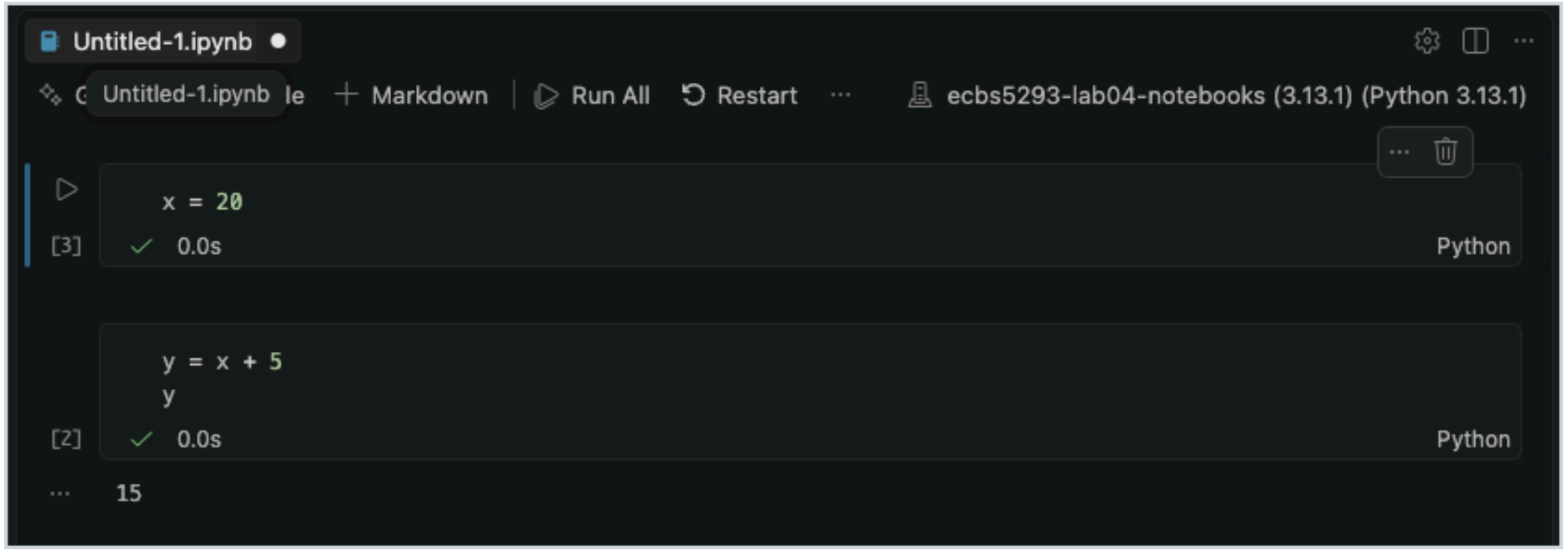
what it remembers right now

```
df = <DataFrame 35 rows>
```

```
total = 28317.25
```

Delete cell 1 from the page. df is still in the kernel. Cell 2 still runs. Nothing on the page says why.

Predict: what is `y`?



Cell 1 said `x = 10` when cell 2 ran. Then cell 1 was edited to `x = 20` and run **alone**. Write down what `y` is in the kernel right now. Do not run anything.

Reveal: the page says 25. The kernel says 15.

The page (.ipynb)

```
[3]  x = 20                                   edited, then run again

[2]  y = x + 5
     y
     15
```

The kernel (a running process)

what it remembers right now

```
x = 20
y = 15
```

y is still 15. Nothing re-ran cell 2. The page reads x = 20, y = x + 5: 25. The kernel remembers 15.

Ask the kernel what it remembers

```
In [1]: x = 10

In [2]: y = x + 5

In [3]: x = 20

In [4]: %whos
Variable   Type    Data/Info
---------------------------
x          int     20
y          int     15

In [5]:
```

`%whos` in any cell prints the blue box: every name the kernel holds, with its type and value. Not what the page says. What the process has.

Variables that exist only because of earlier runs

- You renamed `sales` to `df` in cell 2. Cell 7 still says `sales`. It still works.
- Because `sales` is still in the kernel, from before the rename.
- Save, close, reopen tomorrow, run cell 7: `NameError: name 'sales' is not defined`

The notebook did not break overnight. It was broken this afternoon. The kernel was covering for it.

Imports after first use

```
# cell 4
import pandas as pd
```

- `pd` was first used in cell 1. It worked, because you ran cell 4 last week.
- Fresh kernel, top to bottom: cell 1 fails with `NameError: name 'pd' is not defined`
- Rule: **all imports in the first cell.** Then a fresh kernel cannot lie about them.

Saved outputs are a screenshot of the past

The page (.ipynb)

[3]
```
summary = df_clean.groupby("region")["revenue"].sum()
East 4812.25  North 3990.50 …
```

[1]
```
df_clean = df.dropna(subset=["units"]).copy()
(35, 6)
```

[2]
```
df = pd.read_csv(PROJECT_ROOT / "data" / …)
(40, 5)
```

The kernel you just opened

what it remembers right now

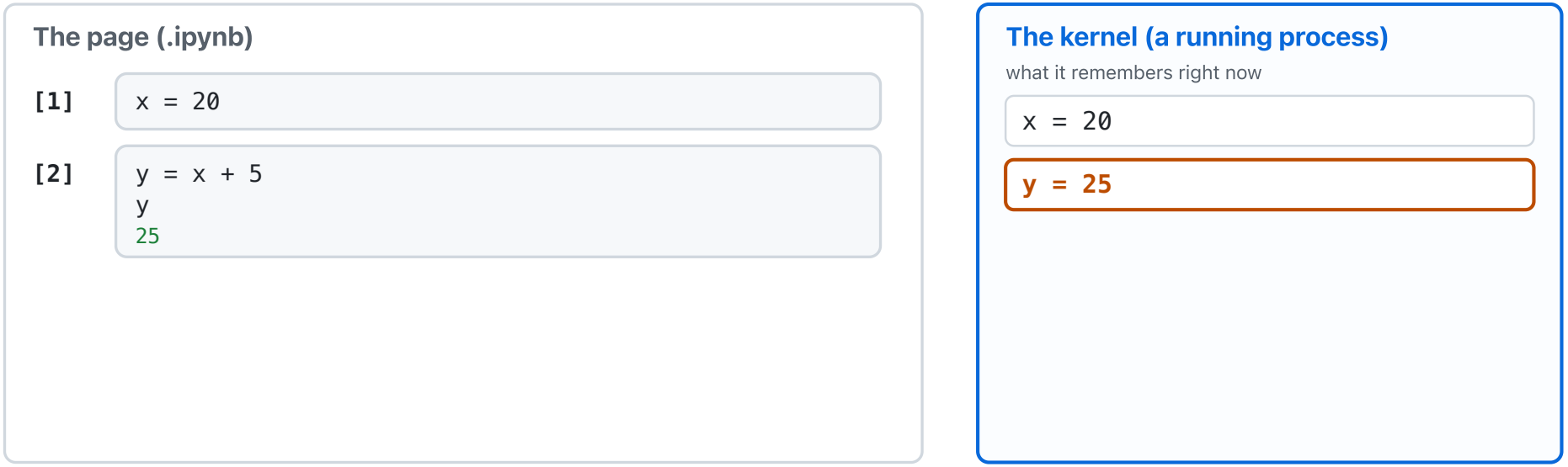
nothing

a fresh kernel knows only what the page defines, once it runs it

Every cell shows an output. The kernel that produced them is gone. These outputs are a screenshot of the past.

The truth test

Restart Kernel and Run All



Restart Kernel and Run All: memory wiped, every cell in page order. Now the page and the kernel agree: 25.

VS Code: notebook toolbar ··· → **Restart Kernel and Run All Cells** (or **Restart**, then **Run All**). JupyterLab: Kernel menu → Restart Kernel and Run All Cells. Kills the process; every variable and import is gone; runs every cell top to bottom from nothing. If it finishes clean, the page and the kernel agree. If it fails, the page was lying. Clean says nothing about whether the numbers are right.

The same test, from the terminal

```
anna@laptop project $ uv run jupyter nbconvert --to notebook --execute demo.ipynb 2>&1 | tail -14
------------------
y = x + 5
y
------------------


---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[1], line 1
----> 1 y = x + 5
      2 y

NameError: name 'x' is not defined

anna@laptop project $
```

`nbconvert --execute` starts a fresh kernel and runs every cell in page order, no clicking. It is how the lab checks run. The traceback reads like any other: bottom line first, then the cell it names.

Notebook vs script

- **Notebook** — exploring. Look at a dataframe, try a plot, change your mind. State is the feature.
- **Script** — repeating. Same input, same steps, same output, every time. No state survives.
- A script cannot carry kernel state between runs; it starts from nothing every time. Its *inputs* can still change its output. For this course's scripts: same input, same output.

When a notebook has stopped changing and you just need it to run again tomorrow, that is a script.

Minimal reproducibility

The only question: **can I restart and run it again?**

- Fresh kernel, or a fresh `uv run python scripts/analyze.py`, on a fresh `uv sync`
- Same output as last time, from the code as it is on the page, right now
- Not "did it ever work." Not "does the chart look right." *Will it work again, from nothing?*

This is the standard for every deliverable in this course from today on.

Where the kernel comes from — the rule, again

Open the **project folder** in VS Code → kernel picker → the `.venv/` interpreter.
(Or `uv run jupyter lab` from the folder; verify the same way.)

- A global kernel, Anaconda, or another project's `.venv` : **right code, wrong Python**
- The cells are correct. The kernel searches a different `sys.path` . `ModuleNotFoundError` .
- First check in any confused notebook: `import sys; print(sys.executable)` in a cell

Three things a kernel has that the page does not show

Hidden property	Session	The question	The check
a working directory	1	*Where am I?*	`Path.cwd()` in a cell
an interpreter	2, Block 3	*Which Python?*	`sys.executable` in a cell
a memory	2, Block 4	*In which state?*	`%whos`; then **Restart and Run All**

Every notebook failure in this course is one of these three. The page shows none of them.

A whiff of Git — checkpoints, and what not to commit

Good commit messages

```
anna@laptop project $ git log --oneline
52481bb (HEAD -> main) Move imports to first cell; notebook passes Restart and Run All
62995f9 final2
72132e7 final
ebbe0db update
a01ce17 fix
691c834 Starter
anna@laptop project $
```

- Say **what changed and why it is now correct**. Present tense. One line.
- A message is for the person reading `git log` in six weeks. That person is you.

The message should let a reader decide whether to look at the diff without looking at the diff.

Commit early, commit ugly

```
anna@laptop project $ git status
On branch main
nothing to commit, working tree clean
anna@laptop project $ echo "oops, I deleted half the cells" > notebooks/clean_and_plot
.ipynb
anna@laptop project $ git status --short
 M notebooks/clean_and_plot.ipynb
anna@laptop project $ git restore notebooks/clean_and_plot.ipynb
anna@laptop project $ git status
On branch main
nothing to commit, working tree clean
anna@laptop project $
```

- Your history is **for future-you, not an audience**. Five ugly checkpoints beat one polished commit, because checkpoints are what `git restore` can reach.
- One thing to *recognize*, not learn: `git status` starts with `On branch main`. A branch name you don't recognize there? **Stop and ask a TA.**

Why your notebook is always "modified"

- A `.ipynb` is a text file of JSON: the code, **and** every saved output, **and** every bracket number
- Restart and Run All rewrites the outputs and renumbers the brackets. Save, and `git status` says `M`, even if you changed no code.
- `git diff` on a notebook is a wall of JSON. Read `git status` for *whether*; open the notebook for *what*.
- So: run clean → save → commit. The outputs in the commit are the ones a fresh kernel produced.

`.gitignore` — what it is

- A plain text file at the repo root, listing paths Git should **skip**
- Ignored paths are skipped by `git status` and by `git add .`, so the ordinary accident cannot happen
- Two limits: a file Git already tracks stays tracked, and `git add -f` forces one in. A guard, not a law.
- Every course repo ships the same standard one. You will read it, not write it.

Git is for source work and meaningful checkpoints, not local junk.

The file that ships in every course repo

```
# Python
__pycache__/
*.py[cod]
*$py.class
.Python
# Virtual environments
.venv/
venv/
env/
ENV/
# Jupyter
.ipynb_checkpoints/
*-checkpoint.ipynb
```

```
# IDE / editor
.vscode/
.idea/
*.swp
*.swo
.DS_Store
Thumbs.db
# Course-specific: generated output
output/
*.log
# Secrets — never commit these
.env
*.key
credentials.json
```

1. **Python:** compiled bytecode; Python regenerates it every run
2. **Virtual environments:** hundreds of MB; uv sync rebuilds it
3. **Jupyter:** autosave copies of your notebooks: stale duplicates
4. **IDE / editor:** your editor's and OS's private notes
5. **Course-specific:** if the code produces it, rerun the code; not source
6. **Secrets:** a committed key is public forever

Not in the file, on purpose: data/. Course data is small and committed with the code. Do not add that line.

Predict: the guard, tested

- Last week, before this file existed, you ran `git add .` and committed. `.venv/` went in.
- Today `.venv/` is in `.gitignore`. You run `uv sync`, which rewrites files inside `.venv/`.
- **What does `git status` show now?** Write one line. Do not type.

AI thread — three good uses today

- *"Here is my notebook. Which cells depend on state from cells that run later? Where are the hidden-state risks?"*
- *"Restart and Run All fails at cell 3 with this* `NameError`*. Here is the cell order and the traceback. Why?"*
- *"These four cells repeat the same cleaning steps. Help me turn them into one function."*

Give it the actual cell order and the actual traceback. Then restart and run it yourself.

AI can clean up code. Restart and Run All is the truth test.

- The assistant can reorder cells, name things better, and write the function. Good.
- It cannot run your kernel. It does not know what your kernel remembers.
- Every AI suggestion in a notebook ends the same way: **Restart Kernel and Run All.** Clean or not clean.

That is the third time. It will be on the exam.

5-minute stand-and-stretch

Stand up. Leave the laptop. Back in five.

Lab 4 — The notebook lies

```
git clone <the Lab 4 starter>
cd <the folder it created>
uv sync
```

- Open the **folder** in VS Code; open `notebooks/clean_and_plot.ipynb`; kernel → `.venv/`; verify with `sys.executable`
- The saved outputs look correct. Every result is there. **Do not trust them.**
- **Restart Kernel and Run All Cells.** It fails. Read the error, note the cell it names, and work out why that cell cannot run from a fresh kernel.
- Fix it so the page tells the truth top to bottom. Then commit, and do the recovery drill: break the notebook, `git status`, `git restore`, run clean again.

What you produce, and the checkoff

1. The notebook passes **Restart Kernel and Run All Cells** — clean, top to bottom, twice
2. Imports in the first cell; no cell depends on a later one
3. One commit with a message that says what you fixed; `git status` clean afterward, no `.ipynb_checkpoints/` or `.venv/` in it
4. Recovery practised: a deleted cell brought back with `git restore`, then a clean run again
5. `DIAGNOSIS.md`: symptom, cause, evidence (the `NameError` line, the cell numbers), change, verification
6. Checkoff: explain it to a TA in about a minute. No notes. Then one what-if question about kernel state.

Stretch: repeated cleaning steps into one function; a script that runs the core workflow fresh; one sanity check. Resets in the README; both destroy work.

Homework 2 assigned

Python execution, environments, notebooks, and Git checkpoints — 15% of the grade, 6–7 hours. Due the **Friday after this session, 23:59** — slot on Moodle.

A repo that fails for environment reasons *and* notebook execution-order reasons. Deliver:

- working setup instructions
- dependency file (`pyproject.toml` + `uv.lock` , via `uv add`)
- a notebook that passes Restart Kernel and Run All
- **two** meaningful commits — one after the environment fix, one after the notebook fix — with real messages
- a diagnosis note per failure, with the **reconciliation**: script table = notebook numbers, to the cent; selection = the README's threshold
- `RECOVERY.md` : break a committed file on purpose, `git restore` it, prove it
- a **60–90 second video**: Restart and Run All succeeding, then what broke, what changed, how you verified